AgileX Scout Mini in ROS 2

Autonomous navigation, wheel-slip simulation and slip control

Project explanation guide: robot, sensors, algorithms, experiments and how to present them



The slip world in Gazebo: the blue areas are the three low-friction patches.

Stack	**ROS 2 Jazzy, Gazebo Harmonic, Nav2, robot_localization, slam_toolbox, PyQt5 Ground Station**
Report	ROS_Report_Review2_Sailakshmi.pdf (41 pages); this guide is the spoken companion to it
Code branch	slip-simulation (GitHub)

0. How to use this guide

Each section has two parts: the explanation, and a green **What to say** box with the sentences to speak. Numbers come from the report and the code. Where a number is only a published figure or is unverified, the text says so. Suggested time: 12-15 minutes in total.

Section	Topic	Time
1	Project in one minute	1 min
2	System architecture	1 min
3	Robot and actuators	1 min
4	Sensors	2 min
5	Algorithms in the navigation stack	2 min
6	Slip world	1 min
7	Slip controller (main contribution)	3 min
8	Ground Station	1 min
9	How to run	0.5 min
10	Experiments and results	2 min
11	Problems, limits, questions	2 min

1. The project in one minute

The AgileX Scout Mini is a four-wheel skid-steer mobile robot. The project simulates it in Gazebo inside a 20 m x 16 m civil-department building with six rooms, and makes it navigate on its own with ROS 2 and Nav2.

- **Review 1:** SolidWorks model converted to URDF, facility and occupancy map generated, localisation (EKF + AMCL), Nav2 navigation, a Ground Station GUI, one recorded 23-leg patrol (23/23 legs, mean error 0.25 m). The floor always gave good grip.
- **Review 2 (this review):** a floor with slippery patches, a slip controller that stops the robot from losing its position, direct shortest-path goals from the Ground Station, and nine measured patrol runs.

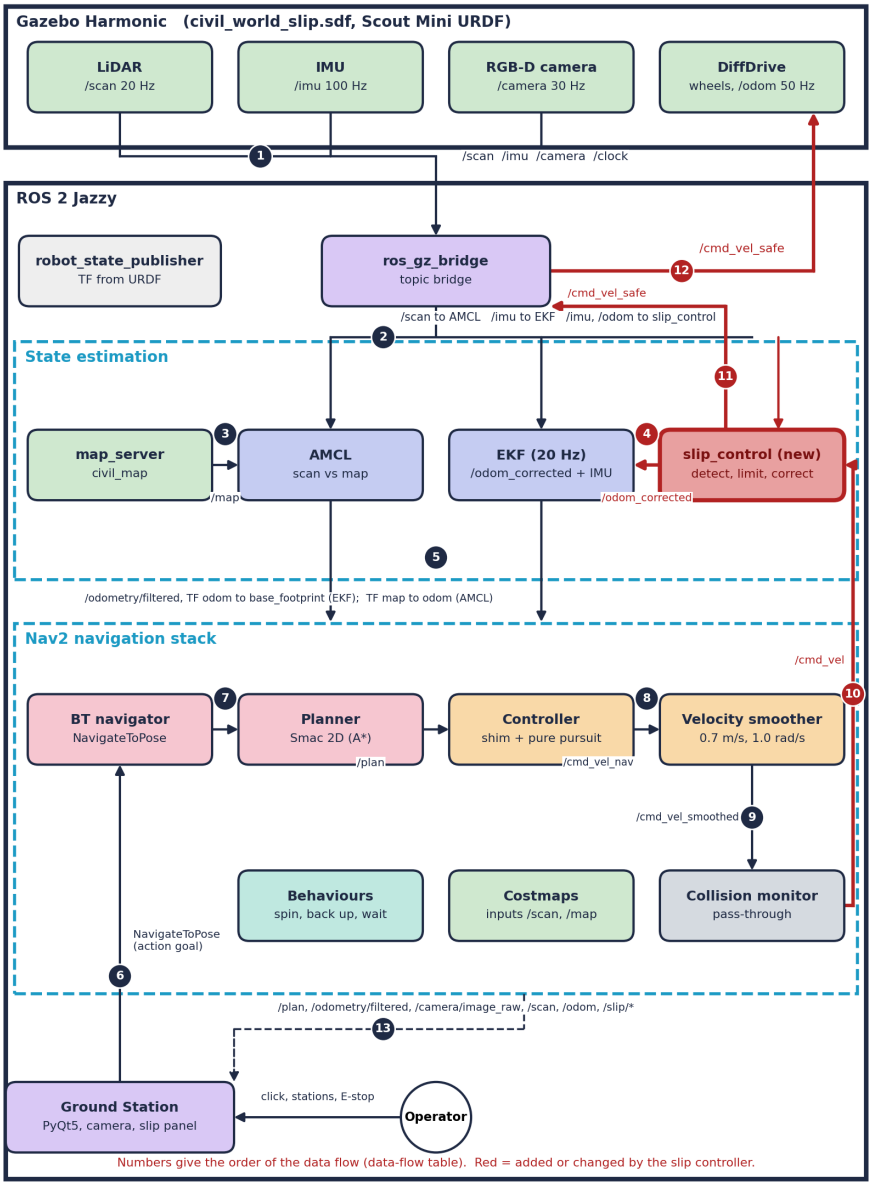
The problem. On a slippery floor the wheels spin. Wheel odometry then reports distance that was never travelled, the localisation filter believes it, and the robot ends up in the wrong place.

The idea. Do not trust the wheels alone. Estimate the real ground speed from the IMU, compare it with the wheel speed, detect slip, slow the wheel command down so the wheels regain grip, and correct the odometry before the EKF sees it.

> **What to say:** In the first review the robot assumed perfect grip. In this review I removed that assumption: I built a slip world, a slip controller that uses the IMU to find the true speed, and I measured the result in nine runs.

2. System architecture

Everything is a ROS 2 node, connected by topics. The command path is the key to understanding the slip controller: Nav2 writes **/cmd_vel**; slip_control reads it, limits it and publishes **/cmd_vel_safe**; the bridge sends that to Gazebo. On the way back, slip_control publishes **/odom_corrected**, which the EKF uses instead of the raw odometry.



Architecture with the slip controller (red marks what is new). Numbers give the order of the data flow.

> **What to say:** Sensors in Gazebo go through the bridge to ROS. State estimation (map, AMCL, EKF) tells the robot where it is. Nav2 plans and drives. My slip controller sits between Nav2 and the wheels, and between the odometry and the EKF.

3. The robot and its actuators

The simulated robot is the Scout Mini model from SolidWorks, converted to URDF: 28 kg, four driven wheels, radius 0.08 m, track 0.612 m. A Gazebo **DiffDrive** plugin treats the skid-steer base as an equivalent differential drive: it takes forward speed v and turn rate ω and turns them into wheel speeds.

Item	Value
Wheel joints	j1, j2 left side; j3, j4 right side; continuous joints, effort limit 40, velocity limit 25 rad/s
Plugin limits	+-2.5 m/s linear, +-3.5 rad/s; acceleration +-4 m/s2 and +-6 rad/s2; stops if no command for 0.5 s
Wheel friction	mu = 0.25 (floor normally gives grip; patches have mu = 0.04)

Item	Value
Example	Straight at 0.6 m/s: wheels turn at v/r = 7.5 rad/s. Turn at 0.6 m/s and 1.0 rad/s: outer wheels 11.3 rad/s, inner 4.0 rad/s

Important honesty point. The simulation has no motor current, torque curve, gearbox or battery model: the wheels follow the command rigidly. That is why slip must be created by lowering the floor friction. The real Scout Mini figures (4 x 150 W brushless motors, 24 V battery, about 10 km/h) are published values from the manufacturer and resellers and were not measured on a physical robot.

> **What to say:** The robot is controlled only through speed and turn rate. The motors are not modelled electrically, so I create slip by making the floor slippery, not by the motor.

4. Sensors: what each one measures and why it is there

Sensor	Topic / rate	What it measures	Used for
2D LiDAR	/scan, 20 Hz	720 range samples over 360 degrees, 0.12-16 m, noise 0.01 m	AMCL localisation, costmaps (obstacles), SLAM mapping
IMU	/imu, 100 Hz	3-axis gyro (noise 0.0002 rad/s) and accelerometer (0.017 m/s2)	Yaw rate for the EKF; acceleration for the slip estimator
Wheel odometry	/odom, 50 Hz	Speed and pose from the commanded wheel motion	EKF input; compared with the IMU to find slip
RGB-D camera	/camera, 30 Hz	640 x 480 colour image, depth 0.1-10 m	Ground Station view only; not used by Nav2

Why this set. LiDAR gives absolute position information against the map. The IMU is independent of the wheels, so it still tells the truth when the wheels spin. Wheel odometry is accurate on good floors and wrong on slippery ones, so the disagreement between wheel odometry and IMU is exactly the slip signal. The IMU yaw rate is also fused because wheel-derived yaw drifts on a skid-steer base (only 60-65 % of the commanded yaw was measured before the IMU was fused).

Sensor fusion. The EKF (robot_localization, 20 Hz, 2D) fuses forward speed from /odom_corrected and yaw rate from the IMU and publishes odom to base_footprint. AMCL (particle filter, 1000-4000 particles) matches /scan to the map and publishes map to odom.

> **What to say:** Wheels can lie on ice, the IMU cannot. Comparing the two is how I know the robot is slipping. LiDAR keeps the pose anchored to the map.

5. Algorithms in the navigation stack (existing, used unchanged)

Algorithm	In one line	Why / key setting
slam_toolbox	Builds the occupancy map from LiDAR while driving	Used once in Review 1 to make the map
AMCL	Particle filter; each particle is a pose guess scored against the LiDAR scan	Likelihood-field model, ignores returns closer than 0.55 m (the robot body)
EKF	Extended Kalman filter combining speed and gyro	Smooth, drift-limited odom frame; 20 Hz
Smac 2D planner	A* search on the costmap grid for the shortest path	cost_travel_multiplier 1.0 so it minimises distance

Algorithm	In one line	Why / key setting
RotationShim + Regulated Pure Pursuit	Rotate to face the path, then chase a look-ahead point on it	Cruise 0.6 m/s; yaw goal tolerance 3.14 rad (position only)
Velocity smoother	Limits acceleration of the command	Cap 0.7 m/s
Collision monitor	Last-line stop or slow-down from LiDAR points	Safety layer
Costmaps	Global and local grids with inflated obstacles	Keep the 0.68 m wide robot clear of walls and posts

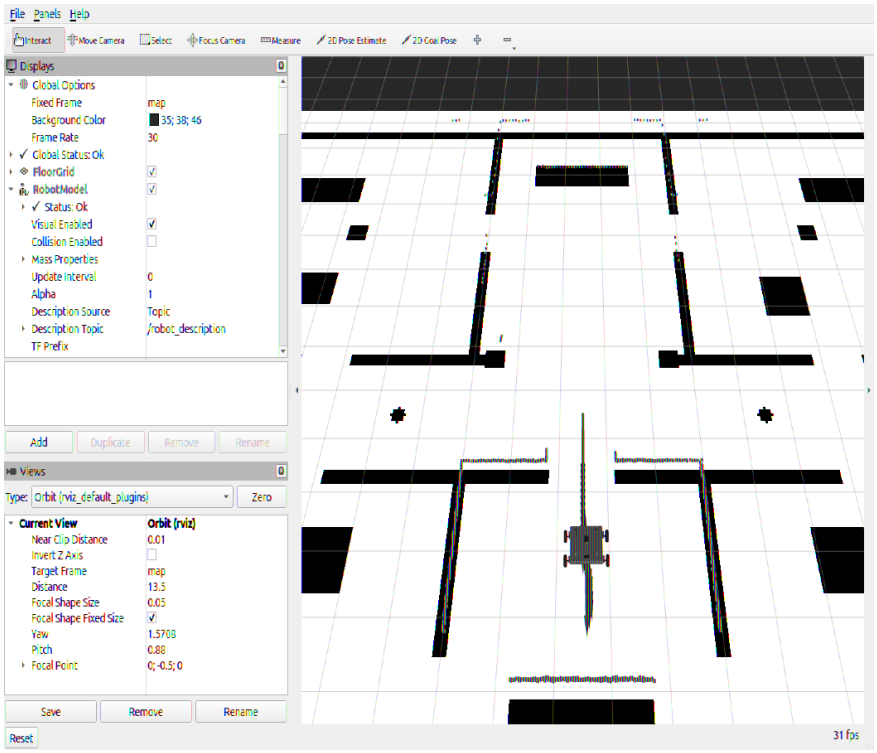
What to say: Nav2 does the planning and path following. I did not change its algorithms; I changed its parameters for shortest-path behaviour and added the slip layer around it.

6. The slip world

civil_world_slip.sdf (world name amr_world) is the Review 1 building with three flush patches of friction mu = 0.04 against the wheel's 0.25: the north doorway (0, 2.3) 1.2 x 1.8 m, the east corridor (1.1, 0) 1.0 x 2.4 m and the west corridor (-1.1, 0) 1.0 x 2.4 m. They are flush with the floor (a version with raised tiles stopped the robot at the 3 mm edge), so the robot and the map cannot see them. Slip has to be detected from motion.



Top view of the slip world (Gazebo camera render): blue = low-friction patches.

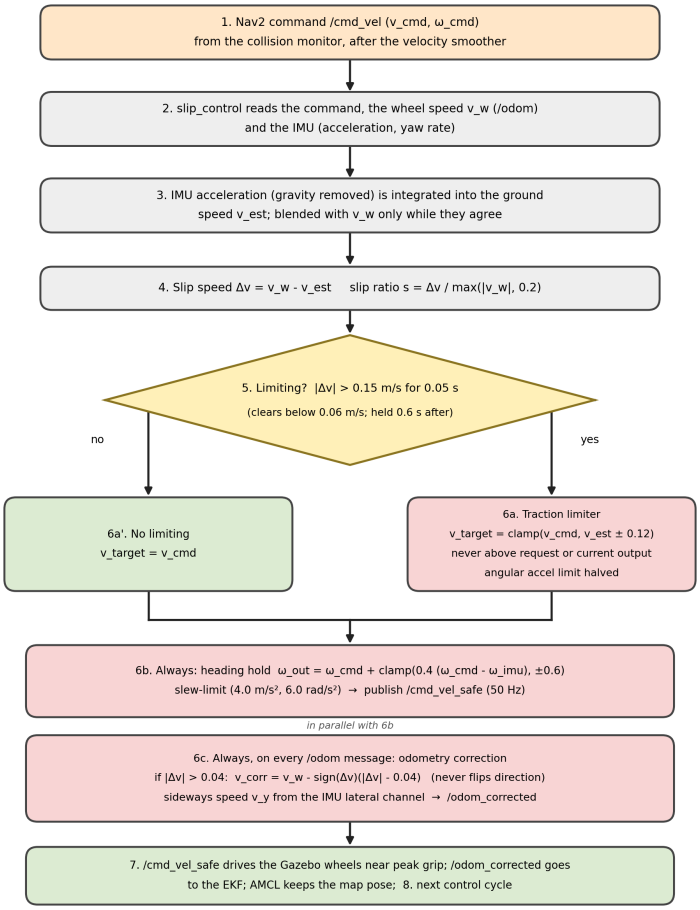


RViz2 view in the same world: map and costmap in black, LiDAR scan, robot, Nav2 plan. The patches are not drawn, as they are only a friction change.

> **What to say:** The patches are invisible to the robot's sensors. That is realistic: wet floor does not show on a LiDAR scan.

7. The slip controller (slip_control.py): the main contribution

It runs as a ROS 2 node. It reads /cmd_vel, /odom and /imu, and publishes /cmd_vel_safe and /odom_corrected. Detection runs at the IMU rate (100 Hz), the command limiter at 50 Hz.



Control loop of slip_control.

Step 1: ground-speed estimate. Integrate the forward IMU acceleration to get v_est. Because integration drifts, it is blended with the wheel speed in a complementary filter, but only when the two agree about speed and acceleration (a gate), so a spinning wheel cannot drag the estimate along. Protections: a zero-velocity update when the robot is quiet, a latch breaker for slow IMU drift, and an energy bound (v_est cannot exceed the recent peak wheel speed by more than 0.05 m/s).

Step 2: slip detection. slip speed = v_wheel - v_est, ratio = slip speed / max(|v_wheel|, 0.2). Slip turns on when |slip speed| > 0.15 m/s for 0.05 s and turns off below 0.06 m/s (hysteresis, so it does not flicker).

Step 3: traction limiter. While slipping, and for 0.6 s afterwards, the speed command is clamped to v_est +- 0.12 m/s, never above the request and never above the current output. Like a car's traction control: stop feeding the spinning wheels more speed so they regain grip. Angular acceleration is halved while limiting.

Step 4: yaw-rate feedback (always on). w_target = w_cmd + clamp(0.4 (w_cmd - w_gyro), +-0.6) keeps the heading straight when one side slips more.

Step 5: odometry correction. The wheel speed is reduced toward v_est once the difference exceeds a dead band of 0.04 m/s, a lateral velocity is estimated from the IMU (with the centripetal term removed), and the pose is integrated with IMU yaw. The result is published as /odom_corrected; the EKF uses it instead of /odom.

```
slip_speed = v_wheel - v_est
if |slip_speed| > 0.15 for 0.05 s:  slip = True
if |slip_speed| < 0.06:             slip = False
limiting = slip or (now - t_clear) < 0.6
if limiting: v_out = clamp(v_cmd, v_est - 0.12, v_est + 0.12)
```

```
publish /cmd_vel_safe;  publish /odom_corrected
```

Topic	Meaning
/slip/active	True while slip is detected
/slip/ratio	Slip ratio, -1 to 1
/slip/traction_scale	Output speed / requested speed (e.g. 64 % in the report example)
/slip/ground_speed_est	The IMU-based speed estimate
/slip/correction_enabled	Latched switch (slip_correction:=true/false)

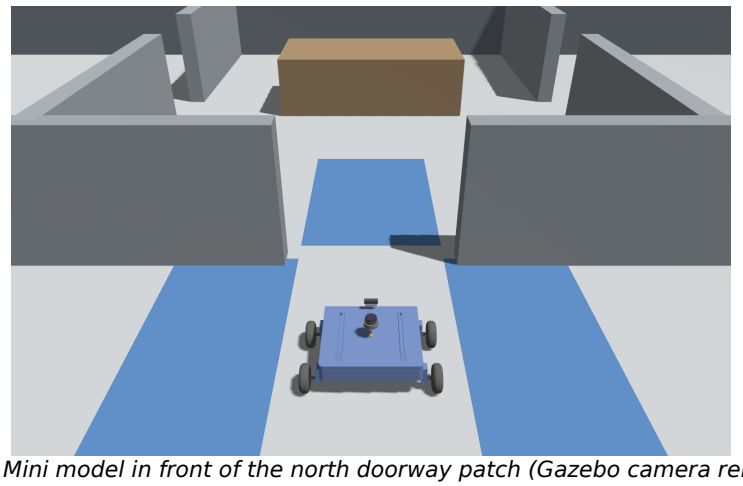
Honest notes. The parameter acc_slip (0.8 m/s2) is declared in the code but not used. Correction ON/OFF is chosen at launch, which is how the comparison runs were made.

> **What to say:** The controller does three jobs: estimate the true speed from the IMU, limit the wheel command when the wheels run ahead of it, and give the EKF a corrected odometry. The comparison between ON and OFF is the evidence that it helps.

8. The Ground Station and goal management

A PyQt5 application with a camera view, a map, telemetry (wheel speed, ground speed, slip %, traction %), room buttons and an emergency stop. Clicking a room or the map sends **one** NavigateToPose goal to Nav2.

- **Pre-emption:** each click increments a route_id; an older route notices it was superseded and stops without cancelling the new goal, so one click is enough.
- **Goal snap:** a clicked point is moved to the nearest cell at least 0.55 m from obstacles, so Nav2 does not reject it.
- **Near-goal acceptance:** within 0.30 m of the goal for 6 s counts as reached.
- **Review 1 versus 2:** Review 1 split each click into door-aligned waypoints, which forced detours. Review 2 sends the direct goal and lets Smac 2D find the shortest path. The driven tours were within +1.8 % (normal) and +2.2 % (slip) of the shortest grid path.



Scout Mini model in front of the north doorway patch (Gazebo camera render).

Why the patrol has 20 legs, not 23. The three removed legs were Review 1 via-points in the corridor. With direct goals and shortest-path planning they added stops and turns without changing the rooms visited, so they were removed. Results must not be compared leg-for-leg with the old 23-leg run.

> **What to say:** One click gives one goal. The planner finds the shortest path, the Ground Station only handles goal snapping, cancelling old routes and the emergency stop.

9. How to run it

```
cd ~/agilex && colcon build --symlink-install
source install/setup.bash
# simulation + Nav2 + RViz2 + slip world, correction ON
ros2 launch assem2_robot civil_simulation.launch.py use_nav:=true \
     world:=<install>/share/assem2_robot/worlds/civil_world_slip.sdf slip_correction:=true
# correction OFF for comparison: slip_correction:=false
./run_civil_nav.sh                      # normal floor with Ground Station
ros2 run assem2_robot civil_nav2_mission # 20-leg patrol with ground-truth checks
```

Always start the simulation fresh: the robot must begin at the world origin. Use colcon build --symlink-install so the installed scripts follow the source (a stale installed copy once ran the old 23-leg script).

> **What to say:** Two launch arguments matter: the world file, and slip_correction true or false.

10. Experiments and results

Method. The 20-leg patrol (three rooms with stations, corridor lanes, return to the hub) was run under three conditions. Each arrival was checked against Gazebo ground truth: position tolerance 0.45 m, heading tolerance 40 degrees at stations, 3 retries, 120 s per leg.

Condition	Outcome	Notes
Normal floor	4 of 4 runs completed 20/20 legs	Worst position error 0.37 m
Slip floor, correction ON	3 of 4 completed	One stop at leg 9: heading 45-46 degrees against the 40-degree limit; position error only 0.14 m
Slip floor, correction OFF	2 of 3 completed	One stop at leg 4; overshoot of 0.48-0.61 m; the robot overshot the corridor hub in all three runs

- Route efficiency: +1.8 % (normal) and +2.2 % (slip) over the shortest grid path.
- Station heading error: worst 38 degrees of the 40-degree limit.

Reading the result. With the correction the robot finished more runs and did not overshoot; its one failure was a heading-tolerance miss, not a localisation failure. Without the correction, overshoot occurred in every run. The sample is small, so call it a clear trend, not a statistic.

> **What to say:** The correction did not make the robot perfect, but it removed the overshoot and the position errors. With only a few runs per condition, I describe this as evidence of a trend.

11. Problems solved, limitations, likely questions

Problems solved (pick three).

- Raised patches stopped the robot at the 3 mm edge, so patches are now flush with the floor.
- The speed estimate could latch onto the wheel speed or drift, so I added the gate, latch breaker and energy bound.
- Clicks needed several tries or ignored new goals, fixed with direct goals, route_id pre-emption and goal snap.
- Gazebo and RViz screenshots were blocked by Wayland: Gazebo through camera sensors, RViz2 through XWayland window capture.
- The installed patrol script was stale (old 23 legs), fixed with symlink install.

Limitations (state them yourself).

- Simulation only; no real-robot slip test; motors are not electrically modelled.

- Friction is fixed at 0.04 on the patches; real floors vary.
- Nine planned runs plus two extra: indicative, not statistical.
- The leg-12 aborts on the slip floor and the S7 slowdown were not investigated.
- acc_slip is declared but unused.

Likely questions.

Question	Answer
Why the IMU and not just the wheels?	Wheels spin on ice; the IMU measures real body motion.
How do you know it is slipping?	Wheel speed and estimated ground speed differ by more than 0.15 m/s for 0.05 s.
Does IMU integration not drift?	Yes. The complementary filter, zero-velocity update, latch breaker and energy bound control that.
Why did one ON run fail?	The robot stopped on a heading of 45-46 degrees against the 40-degree limit; position was only 0.14 m off.
Is the slip-off result worse?	Yes, it overshot 0.48-0.61 m, but with so few runs I call it a trend.
What next?	Real-robot test, varied friction, and tuning the heading approach near goals.

> **What to say:** I can show what works, what the evidence is, and where it is limited. All numbers are from simulation.